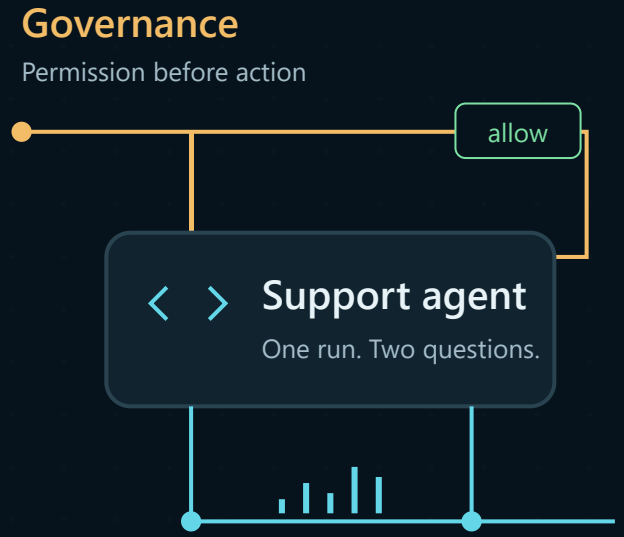
AI Agent Governance and Observability in Practice

Control what the agent can do.

Understand what actually happened.

Xiaodi Yan · Microsoft MVP



Python · one agent · real execution evidence

25 minutes

Would you retry this?

“I couldn’t complete that request.”

The agent’s final response

The action was blocked.

The policy may have worked exactly as intended.

The action failed.

The policy may have allowed it; a dependency timed out.

The same answer can hide two very different runs.

Two capabilities, one application



Governance

May it act?

Policy and permissions

Tool access and approval

Host enforcement



Observability

What happened?

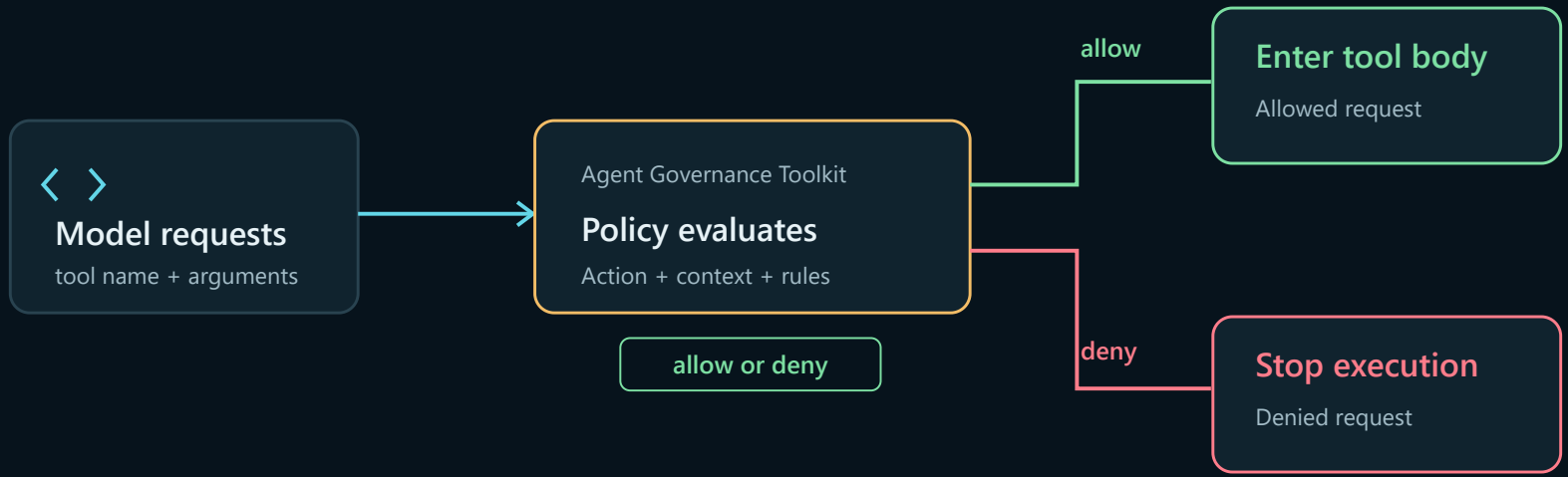
Execution traces

Correlated logs

Latency, tokens and outcomes

Identity, auditing and evaluation belong in the wider system. These two responsibilities work alongside each other.

Control belongs at the action boundary



The host enforces the result

The model proposes. The policy decides. The host enforces.

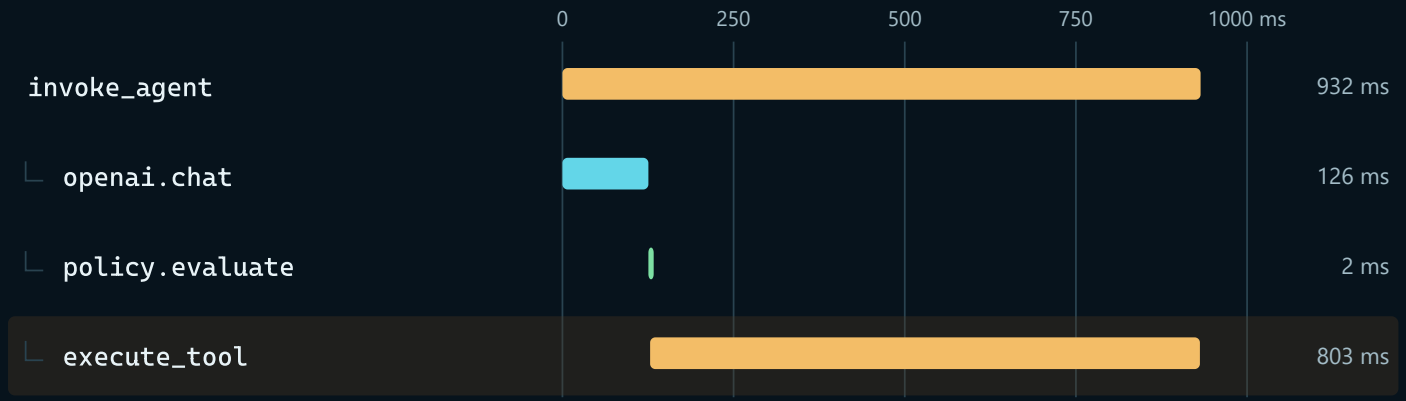
An unmatched action defaults to deny in this demo.

Permission must be checked before the tool body.

Where did the time go?

One trace · injected timeout

932 ms



86% of this run was in the tool body.
Policy allowed it. The tool timed out.

Recorded offline rehearsal. Short spans use a minimum-width marker; durations are printed.

Trace

Locate the slow operation.

Log

Read the rule and error context.

Metrics

Find the pattern across runs.

Execution evidence, not the model's private reasoning.

A common language for AI telemetry

```
// Recorded model-span attributes
{
  gen_ai.operation.name: "chat"
  gen_ai.request.model: "demo-model"
  gen_ai.usage.input_tokens: 80
  gen_ai.usage.output_tokens: 18
}

// Application policy metadata
app.governance.decision: "allow"
```

Conventions describe.

Shared names and meanings.

Instrumentation collects.

OpenLLMetry observes SDK operations.

OpenTelemetry exports.

Context, providers and pipelines.

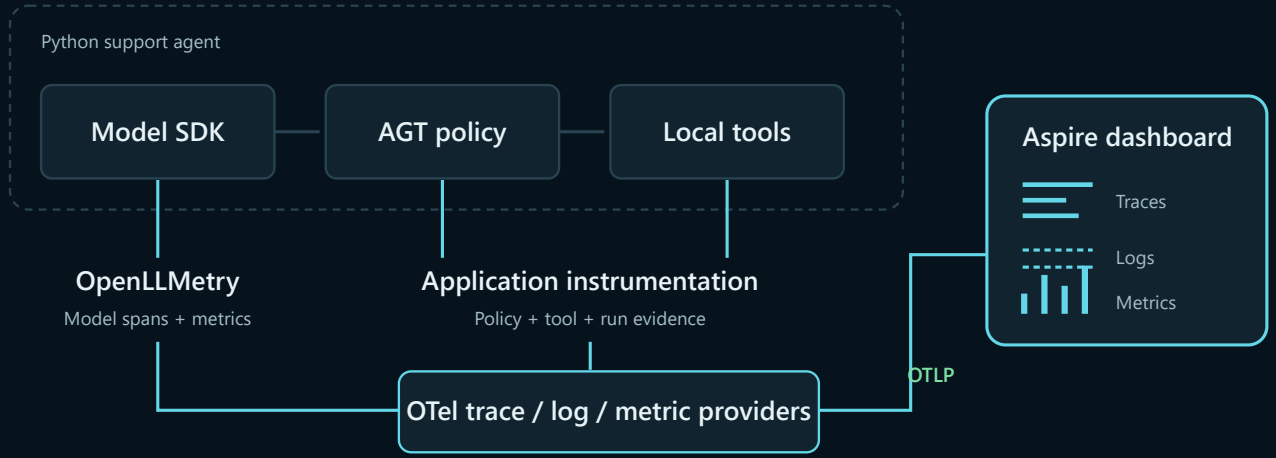
The dashboard visualizes.

Queries, traces and metrics.

GenAI conventions are in Development. Pin your libraries and verify their actual output.

Use app.* for your own business metadata.

Instrument the model. Record the boundary.



Instrumentation collects. OpenTelemetry exports. Aspire visualizes.

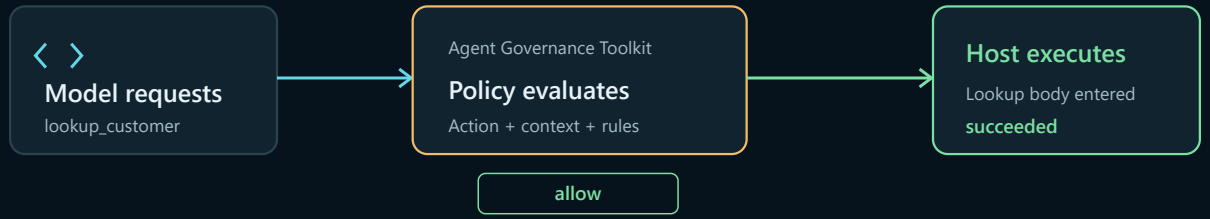
```
os.environ["TRACELOOP_TRACE_CONTENT"] = "false"
OpenAIInstrumentor().instrument(
  tracer_provider=traces, meter_provider=meters)
```

Own your OTel pipeline.
One instrumentation owner per model call.
Traceloop SaaS is optional.

Live demo · 1 Open the support agent

Allowed: the tool executes

> “Look up customer C-1042 and summarize their support status.”



The host enforces the result

1 Lookup tool body entry

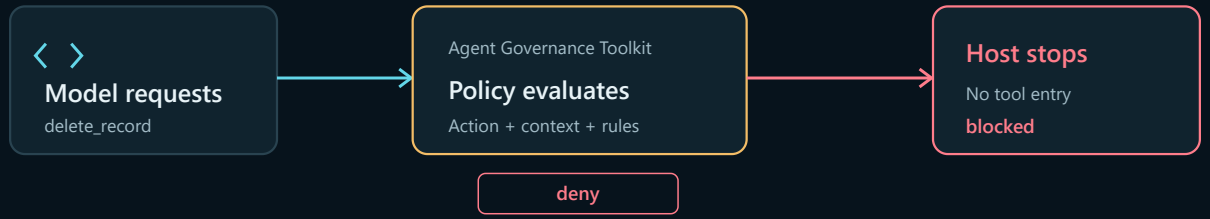
Deletion entries: 0

Two model calls: request the tool, then compose the answer. Follow the shared trace ID.

Live demo · 2 Request denied deletion

Blocked: the control worked

> “Delete customer record C-1042.”



The host enforces the result

0 Deletion tool body entries

Rule: block-destructive

The policy span is not ERROR. Verify the execution counter; a missing span alone is insufficient.

Policy deny. Agent blocked. No tool execution.

Live demo · 3 Run lookup with timeout

Allowed does not mean successful

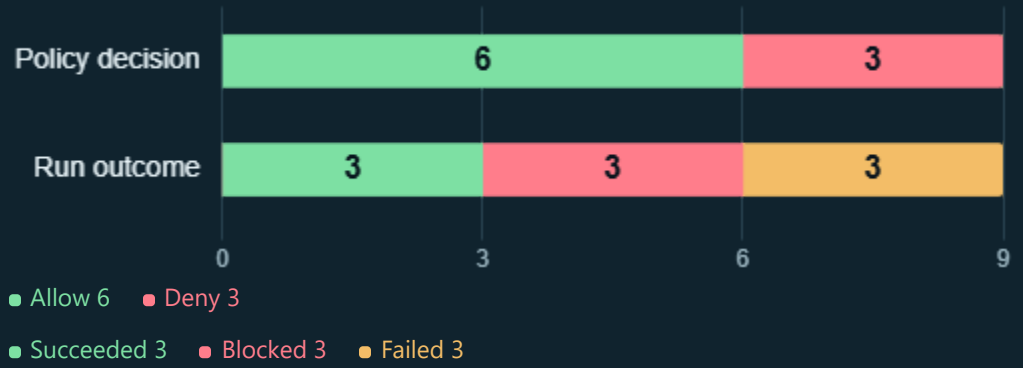
Allowed
Policy decision

Entered
Lookup body: 1

Timed out
Injected 800 ms tool timeout

9 runs. Two different outcome counts.

Recorded offline rehearsal



6 allowed requests ≠ 3 successful runs

1,398 synthetic input + output tokens

Metrics reveal patterns. Traces locate them. Logs explain them.

Make the evidence useful in production

Keep correlation intact.

Preserve context across tools and services.
Connect each policy log to its run.

Capture content deliberately.

Start with model, duration and token usage.
Handle prompt and result capture deliberately.

Keep outcomes separate.

Denied action ≠ failed execution.
Token counts ≠ cost or response quality.

Verify all three pipelines.

Export traces, logs and metrics.
Design durable audit evidence separately.

Aspire standalone is a diagnostic viewer. Production storage, retention and access controls need their own design.

Control the action.

Policy before execution.

Enforce the boundary.

Requested ≠ executed.

Understand the run.

Evidence connected by context.

Connect the signals.

Traces, logs and metrics.

Verify the schema.

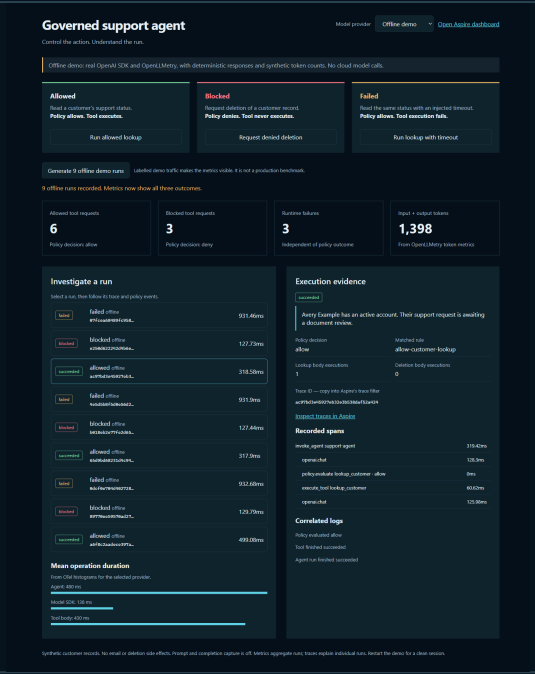
Shared conventions; actual output.

Sources and implementation notes

- Microsoft Agent Governance Toolkit — runtime policy engine and host enforcement.
- OpenLLMetry — Python instrumentations; a separate JS/TS project is also available.
- OpenTelemetry GenAI semantic conventions — spans, metrics, events and provider conventions.
- OpenTelemetry Python — trace, metric and log pipelines.
- Aspire standalone dashboard — OTLP viewer for Python and other languages.

Pinned versions and source checks are in the repository. This demo uses Chat Completions to exercise both automatic model tracing and metrics.

Backup: the actual demonstration



Recorded offline rehearsal. All customer records and model usage counts are synthetic; the telemetry is collected through the real SDK and OpenLLMetry.

Use if a live tab or dashboard is unavailable.

Backup: approvals and standards

Human approval

A policy can require approval.
The host pauses and resumes execution.

Approval must be bound to the action and its arguments.

A resumed request may have a new trace. Preserve the relationship deliberately.

Conventions evolve

Inspect actual emitted fields and units.

Use `app.*` for business policy metadata.

Trace export does not automatically configure metric export.

The newer GenAI draft and a pinned instrumentation may use different metric names.